

Exercises Architecture

1. Name the main components of the M0 CPU and explain their function.

Core Registers	13 x 32-bit registers for temporary storage (r0 – r12) 1 x 32-bit register for Stack Pointer (sp) (r13) 1 x 32-bit register for Link Register (lr) (r14) 1 x 32-bit register for Program Counter (pc) (r15)
ALU	Arithmetic Logic Unit Data processing unit for arithmetic and logic operations
Flags	Processor Status Register: Indicates state of processor.
Control Unit with Instruction Register	Controls execution of an instruction based on the machine code currently stored in the Instruction Register (IR)
Bus Interface	Interface between CPU and external System Bus; Bridge between internal and external bus

2. A processor executes a list of instructions in a predetermined order. Find analogies from everyday life.

Baker/cook following a recipe
Musician who plays from notes (sheet music)
Pilot, who runs through a checklist before takeoff

3. How many memory bytes can be addressed with an 8-bit memory address? How many with a 16-bit address? How many with a 32-bit address?

8 bit → 256 Bytes
16 bit → 64 KBytes = 65'536 Bytes
32 bit → 4 GBytes = 4'294'967'296 Bytes

4. Name 3 instruction types of the M0 CPU.

8 bit → 256 Bytes
16 bit → 64 KBytes = 65'536 Bytes
32 bit → 4 GBytes = 4'294'967'296 Bytes

5.

6. What is the function of the following registers?
a. pc

Program counter: Points to the address where the instructions will next be read

b. sp

Stack pointer: Points to the memory addresses where the elements are written/read from the stack

c. lr

Link Register: Used to keep track of the positions where to jump back (e.g. routines)

7. Why is the PC initialized to a defined value at reset? (Although other CPU registers may have an undefined content.)

Link Register: Used to keep track of the positions where to jump back (e.g. routines)

8. Name the different parts of a line in assembly code.

Label, mnemonic, operands, comment

9. What is a memory map? What is it used for?

It is a graphical layout (map) showing the addresses and sizes of elements that communicate with the CPU (memories, Inputs, Outputs)
The memory map helps users to know where each element is (e.g. when writing the appropriate drivers)

10. How many byte positions can be addressed by the M0? Which positions in the memory map need to be occupied? Explain your answer.

The M0 has a 32-Bit address bus. Therefore, it can address 4 GByte = 2^{32} Bytes. Positions needed for the initialization of the processor (at Boot) must be covered by the proper elements (memory). Otherwise, there will be no correct start.

11. Explain the following terms

a. Fetch

Get the instruction from code memory

b. Execute

Do what the instructions say

c. Word

A 32-bit memory unit (for the M0)

d. Half-word

A 16-bit memory unit (for the M0)

e. Little endian

A multi-byte representation where the LSByte is at the lower address

f. Big endian

A multi-byte representation where the MSByte is at the lower address

g. Word Alignment

The address of the multi-byte element is a multiple of the word length (4 for the M0)

12. A program (code in C) has variables represented as below in the memory map. Determine the decimal values of the variables Var1 Var5. Assume Little Endian representation.

Address	Byte content (decimal, hex, binary)	Variable
0x2FFF'FFF7	0x45	
0x2FFF'FFF8	0xE2	Var4 (16-bit short)
0x2FFF'FFF9	01100010 (binary)	
0x2FFF'FFFA	213	
0x2FFF'FFFB	25	
0x2FFF'FFFC	0x65	Var3 (32-bit unsigned integer)
0x2FFF'FFFD	10101101(binary)	
0x2FFF'FFFE	0xA3	
0x2FFF'FFFF	0x82	
0x3000'0000	0xA2	Var5 (32-bit integer)
0x3000'0001	34	
0x3000'0002	0x54	
0x3000'0003	0xFF	
0x3000'0004	0x92	Var2 (unsigned char)
0x3000'0005	0x03	Var1 (char)

Var1 =

Var1 = 0x03 = 3d (it is 8-bit unsigned)

Var2 =

Var2 = 0x92 = 146d (it is 8-bit unsigned)

Var3 =

Var3 = 0x82A3AD65 = +2'191'764'837d (it is 32-bit unsigned!!)

Var4 =

Var4 = 0x62E2 = +25'314d (it is 16-bit signed)

Var5 =

Var5 = 0xFF5422A2 = -1'1263'326 (it is 32-bit signed)
(-2'147'483'648 + 2'136'220'322)

How would the same variable values be stored on a Big Endian platform? Fill in the table.

Address	Byte content (decimal, hex, binary)	Variable
0x2FFF'FFF7	0x45	
0x2FFF'FFF8	01100010 (binary)	Var4 (short)
0x2FFF'FFF9	0xE2	
0x2FFF'FFFA	213 (or 25)	
0x2FFF'FFFB	25 (or 213)	
0x2FFF'FFFC	0x82	Var3 (32-bit unsigned integer)
0x2FFF'FFFD	0xA3	
0x2FFF'FFFE	10101101(binary)	
0x2FFF'FFFF	0x65	
0x3000'0000	0xFF	Var5 (32-bit integer)
0x3000'0001	0x54	
0x3000'0002	34	
0x3000'0003	0xA2	
0x3000'0004	0x92	Var2 (unsigned char)
0x3000'0005	0x03	Var1 (char)

We are not told if 25 / 213 form a unit or not. Therefore, it can be both ways.

13. Which 3 memory areas (sections) can be differentiated for a program? What will be stored in the individual areas? In which memory type can the areas be located?

CODE	read-only → RAM or ROM Machine instructions, constants
DATA	read-write → RAM Global variables, static variables and heap in C
STACK	read-write → RAM Procedure calls / passing of parameters, local variables

14. Assume that the following memory areas are used when executing a program:

- Code: 0x20000000 to 0x200001FF
- Data: 0x20000200 to 0x200002FF
- Stack: 0x20000300 to 0x200003FF

Draw an appropriate memory map and draw the three sections. Label the first and last addresses for each area. How many storage locations does each of the areas contain?

